Logo-Udesa.jpg

I302 - Aprendizaje Automático y Aprendizaje Profundo

Trabajo Práctico 3: Redes Neuronales

Joaquin Szterensus

12 de mayo de 2025

Ingeniería en Inteligencia Artificial

Resumen

Este trabajo práctico busca comprender el funcionamiento y diseño de una red neuronal para el reconocimiento de 49 caracteres japoneses en imágenes de 28×28 píxeles. Se desarrolló por completo una red neuronal básica utilizando exclusivamente la librería `NumPy`. La red incorpora diversas técnicas de optimización y regularización para mejorar su desempeño y reducir el tiempo de cómputo. Además, se emplearon distintas métricas para evaluar el modelo, como la *accuracy* y la pérdida de entropía cruzada.

La selección de hiperparámetros se realizó mediante una búsqueda en *grid search* sobre un subconjunto acotado de valores preseleccionados. Los valores que ofrecieron mejor rendimiento fueron: *learning rate* = 0.001, optimizador = Adam y tamaño de batch = 32, con una arquitectura de [728,1024,512,49]

Finalmente, se implementó una red neuronal equivalente en `PyTorch` para comparar ambos enfoques utilizando las mismas métricas.

1. Introducción

La clasificación automática de caracteres por imágenes es una tarea clásica en el campo del aprendizaje automático. En este trabajo, nos enfocamos en el reconocimiento de 49 caracteres provenientes de la lengua japonesa, representados con imágenes de 28×28 píxeles.

Con el objetivo de comprender en profundidad el funcionamiento de las redes neuronales, se desarrolló desde cero un modelo utilizando únicamente la biblioteca `NumPy`. A su vez, se incorporaron técnicas de entrenamiento como regularización L2, optimización con ADAM, *early stopping*, *learning rate scheduling* y *dropout*. El modelo recibe un vector de longitud 728, que representa cada píxel de la imagen aplanada. El modelo procesa esta entrada en las capas ocultas y luego genera una salida de longitud 49, el cual cada componente representa la probabilidad de que esa imagen pertenezca a cierta clase.

Finalmente, se utilizó la librería `PyTorch` para construir un modelo equivalente, permitiendo comparar su rendimiento con la implementación propia bajo las mismas condiciones experimentales. Esta comparación no solo permitió evaluar la performance de ambos enfoques, sino también analizar las ventajas prácticas que ofrece un *framework* de alto nivel como `PyTorch` en términos de flexibilidad y eficiencia.

2. Métodos

2.1. Algoritmos

El entrenamiento se realizó mediante el algoritmo de *Backpropagation*, el cual consiste en dos fases: propagación hacia adelante para calcular las activaciones y propagación hacia atrás para obtener los gradientes de la función de costo con respecto a los parámetros. Sea para una capa l la entrada $A^{(l-1)} \in \mathbb{R}^{m \times n_{l-1}}$, los pesos $W^{(l)} \in \mathbb{R}^{n_{l-1} \times n_l}$, los sesgos $b^{(l)} \in \mathbb{R}^{1 \times n_l}$, y la pre-activación:

$$Z^{(l)} = A^{(l-1)} W^{(l)} + b^{(l)},$$

A partir de esta preactivación, se calcula la activación de la capa como:

$$A^{(l)} = f\big(Z^{(l)}\big),$$

donde f es la función de activación. En las capas ocultas se usó ReLU, definida como:

$$\mathrm{ReLU}(x) = \text{máx}(0, x),$$

la cual introduce no linealidad y evita el problema del gradiente desvanecido en capas profundas. En la capa de salida se empleó softmax:

$$\mathrm{softmax}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^{C} \exp(z_j)},$$

donde $z = (z_1, \ldots, z_C)$ es el vector de preactivación (logits) de la salida y C es el número de clases. Esta función transforma los logits en probabilidades, normalizando la salida a un vector que suma 1.

En la fase de retropropagación se utiliza la regla de la cadena para calcular el gradiente de la pérdida con respecto a cada parámetro. Para la capa de salida se define:

$$\delta^{(L)} = A^{(L)} - Y \quad \in \mathbb{R}^{m \times n_L},$$

donde $A^{(L)}$ es la probabilidad predicha por softmax y Y es el vector de etiquetas verdaderas codificadas en one-hot. Para capas anteriores se propaga el error hacia atrás como:

$$\delta^{(l)} = \left(\delta^{(l+1)} W^{(l+1)T}\right) * f'\left(Z^{(l)}\right),$$

donde $*$ representa la multiplicación elemento a elemento, y f' es la derivada de la función de activación. Para ReLU, esta derivada es 1 si $z > 0$ y 0 en caso contrario.

Los gradientes con respecto a los pesos y sesgos de cada capa se calculan mediante:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \frac{1}{m} A^{(l-1)T} \delta^{(l)}, \qquad \frac{\partial \mathcal{L}}{\partial b^{(l)}} = \frac{1}{m} \mathbf{1}^T \delta^{(l)}.$$

Estas expresiones permiten actualizar los parámetros eficientemente utilizando operaciones matriciales sobre todos los ejemplos en paralelo.

2.2. Optimizaciones

Para la optimización se emplearon dos métodos. En descenso de gradiente estocástico (SGD) clásico, los parámetros se actualizan como:

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial W^{(l)}}, \qquad b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial b^{(l)}},$$

donde η es la tasa de aprendizaje. En el optimizador Adam, que adapta la tasa de aprendizaje individual por parámetro, se utilizan los siguientes pasos:

$$\begin{aligned}
m_t &= \beta_1 \, m_{t-1} + (1 - \beta_1) \, g_t, \\
v_t &= \beta_2 \, v_{t-1} + (1 - \beta_2) \, g_t^2, \\
\hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \\
W &\leftarrow W - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon},
\end{aligned}$$

donde g_t es el gradiente actual, y β_1, β_2 son hiperparámetros de suavizado. Adam logra convergencia más rápida y estable en la práctica, especialmente en problemas con datos ruidosos o gradientes escasos.

2.3. Regularizaciones

Para evitar el sobreajuste se aplicaron varias técnicas de regularización. La regularización L2 agrega un término penalizante a la pérdida:

$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \frac{\lambda}{2m} \sum_l \|W^{(l)}\|_2^2,$$

donde λ es el coeficiente de penalización y $\|W\|_2$ es la norma euclídea. Esto incentiva pesos pequeños, reduciendo la complejidad del modelo.

Otra técnica utilizada fue *dropout*, que consiste en desactivar aleatoriamente neuronas durante el entrenamiento. Para una capa con activación $A^{(l)}$, se aplica:

$$\tilde{A}^{(l)} = \frac{A^{(l)} * D^{(l)}}{1-p}, \quad D_{ij}^{(l)} \sim \text{Bernoulli}(1-p),$$

donde $D^{(l)}$ es una máscara binaria. La división por $1-p$ es un factor de corrección para mantener el valor esperado de las activaciones constante entre entrenamiento y validación.

Se implementó además *early stopping* con paciencia de 15 épocas, lo cual detiene el entrenamiento si la pérdida de validación no mejora después de un número de iteraciones. Esta estrategia previene el sobre-ajuste y evita entrenar innecesariamente una vez alcanzado el mejor punto de generalización.

Finalmente, se incorporaron dos estrategias de ajuste del *learning rate* para facilitar la convergencia del modelo:

- **Calentamiento lineal con saturación**: esta técnica incrementa la tasa de aprendizaje linealmente desde 0 hasta un valor máximo $\eta_{\max}$ durante un número fijo de épocas (*warmup*), y luego la mantiene constante. La fórmula utilizada fue:

  $$\eta_t = \begin{cases} \eta_{\max} \cdot \frac{t+1}{T_{\text{warmup}}}, & \text{si } t < T_{\text{warmup}}, \\ \eta_{\max}, & \text{si } t \geq T_{\text{warmup}}, \end{cases}$$

  donde t es la época actual y T_{warmup} el número de épocas de calentamiento. Este enfoque ayuda a evitar inestabilidad en las primeras iteraciones, permitiendo que el modelo se adapte gradualmente a los datos.

- **Decaimiento exponencial**: en esta estrategia, la tasa de aprendizaje disminuye exponencialmente con cada época, siguiendo la fórmula:

  $$\eta_t = \eta_0 \cdot \gamma^t,$$

  donde η_0 es la tasa inicial, γ es el factor de decaimiento (entre 0 y 1), y t la época. Esto permite que el modelo realice grandes actualizaciones al principio y refinamientos más pequeños a medida que se acerca a un mínimo.

Ambas estrategias se utilizaron con el objetivo de mejorar la estabilidad y eficiencia del entrenamiento, ayudando a encontrar mejores mínimos a medida que el modelo converge.

2.4. Métricas

Las métricas utilizadas para evaluar el rendimiento de los modelos fueron **accuracy**, pérdida de entropía cruzada y la matriz de confusión, tanto en el conjunto de validación como en el de prueba.

La **exactitud** (*accuracy*) se define como la proporción de predicciones correctas sobre el total de ejemplos evaluados:

$$\text{Accuracy} = \frac{\text{número de predicciones correctas}}{\text{número total de ejemplos}}.$$

La **pérdida de entropía cruzada** mide la discrepancia entre las distribuciones de probabilidad predicha y la verdadera, siendo especialmente útil en clasificación multiclase:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{m}\sum_{i=1}^{m}\sum_{j=1}^{C} y_{ij}\log(\hat{y}_{ij}),$$

donde m es el número de ejemplos, C el número de clases, y_{ij} es 1 si el ejemplo i pertenece a la clase j (y 0 en caso contrario), y $\hat{y}_{ij}$ es la probabilidad predicha para esa clase. Esta métrica penaliza con mayor severidad las predicciones erróneas con alta confianza. .

2.5. Preprocesamiento

El conjunto de datos fue normalizado dividiendo los valores de los píxeles por 255, de modo que quedaran en el rango $[0, 1]$. Esta normalización se aplicó para evitar gradientes elevados y permitir que el modelo trabaje con valores numéricos más pequeños, lo cual mejora la estabilidad y la convergencia durante el entrenamiento. Posteriormente, los datos se dividieron en un 60 % para entrenamiento, 20 % para validación y 20 % para prueba. No se aplicaron técnicas de *data augmentation*.

2.6. Hiperparametros

Para la selección de hiperparámetros y arquitecturas se realizó una búsqueda en rejilla (*grid search*) sobre un subconjunto acotado de valores posibles. Esta estrategia se adoptó debido al elevado costo computacional asociado al entrenamiento de múltiples modelos. Los valores explorados en el *grid search* incluyeron:

- **Arquitecturas**: `[[784, 1024, 512, 49],[784, 512, 256, 49], [784, 256, 128, 49]]`
- **Tasa de aprendizaje**: `[0.01, 0.001]`
- **Optimizador**: `[SGD, Adam]`
- **Dropout probability**: `[None, 0.5]`
- **Regularización L2** (λ): `[None, 0.01]`
- **Tamaño de batch**: `[None, 32, 64]`

Los mejores hiperparámetros obtenidos para el modelo implementado manualmente fueron: **arquitectura** = [784, 1024, 512, 49], **tasa de aprendizaje** = 0.001, **optimizador** = Adam, **tamaño de batch** = 32 , **sin dropout** y **sin Regularizacion L2**.

Para el modelo implementado en `PyTorch`, los mejores valores fueron: **arquitectura** = [784, 512, 256, 49], **tasa de aprendizaje** = 0.01, **optimizador** = SGD, **L2** = None, **batch size** = None, **dropout** = 0.5.

3. Resultados

3.1. Análisis exploratorio de los datos

Antes de entrenar los modelos, se realizó un breve análisis exploratorio para entender mejor el conjunto de datos. En la Figura 1 se observa la distribución de ejemplos por clase. Si bien la distribución es relativamente uniforme, hay algunas clases con una cantidad significativamente menor de ejemplos, lo cual podría afectar negativamente el desempeño del modelo en esas clases.

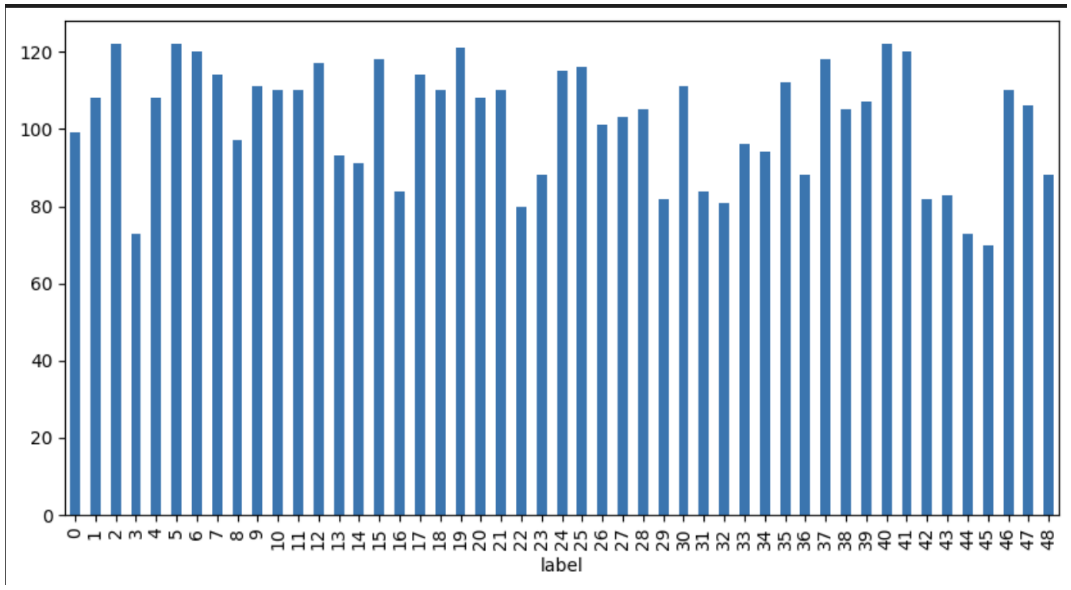

Figura 1: Distribución de ejemplos por clase.

También se visualizaron algunas imágenes aleatorias del conjunto de datos para verificar la claridad y calidad de las imágenes de entrada. En la Figura 2 se presentan tres ejemplos representativos de caracteres japoneses del conjunto.

3.2. Entrenamiento del modelo base (M0)

Como punto de partida, se entrenó una red neuronal básica con dos capas ocultas de 100 y 80 nodos, respectivamente, una tasa de aprendizaje fija de 0.05, y sin técnicas adicionales de regularización ni optimización. Este modelo se denomina **M0**.

En la Figura 3 se observa la evolución de la pérdida de entropía cruzada sobre los conjuntos de entrenamiento y validación a lo largo de 1000 épocas.

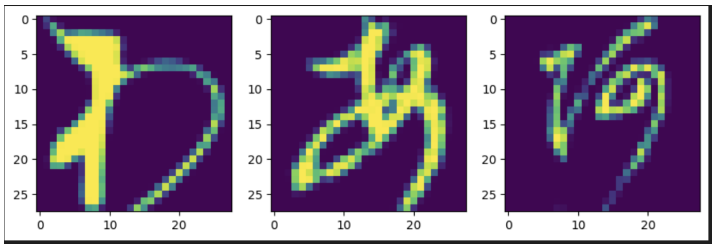

Figura 2: Muestra aleatoria de 3 imágenes de caracteres japoneses.

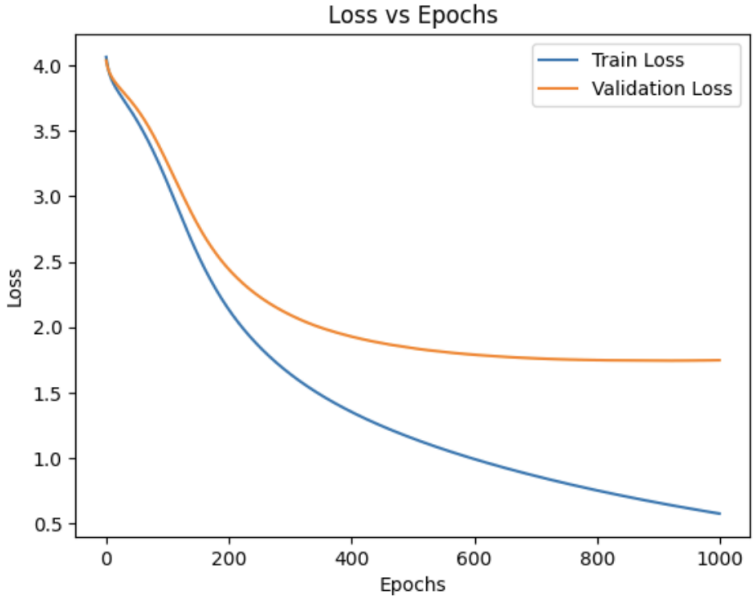

Figura 3: Evolución de la pérdida (cross-entropy) del modelo M0.

Se puede notar que, si bien la pérdida en entrenamiento disminuye de forma constante, la pérdida en validación se estabiliza y deja de mejorar aproximadamente a partir de la época 700. Esto sugiere que el modelo comienza a sobre-ajustar los datos de entrenamiento, un fenómeno común cuando no se aplican técnicas de regularización. Además, la tasa de aprendizaje de 0.05 resultó ser bastante alta, lo cual pudo haber dificultado una convergencia más suave en las primeras épocas.

Cuadro 1: Rendimiento del modelo M0 en validación y entrenamiento

conjunto	**Accuracy (val)**	**Loss (val)**
Validación	57.9 %	1.74
Entrenamiento	- %	-

3.3. Modelo mejorado con regularizaciones e hiperparámetros óptimos (M1)

A continuación se presenta el modelo M1, que incorpora las siguientes mejoras con respecto a M0 y una estructura diferente, [784,512,256,49]:

- **Tasa de aprendizaje**: 0.001
- **Optimizador**: Adam
- **Tamaño de batch**: 32
- **Early stopping**: paciencia 10 épocas

En la Figura 4 se muestra la evolución de la pérdida de entropía cruzada para M1 en entrenamiento y validación.

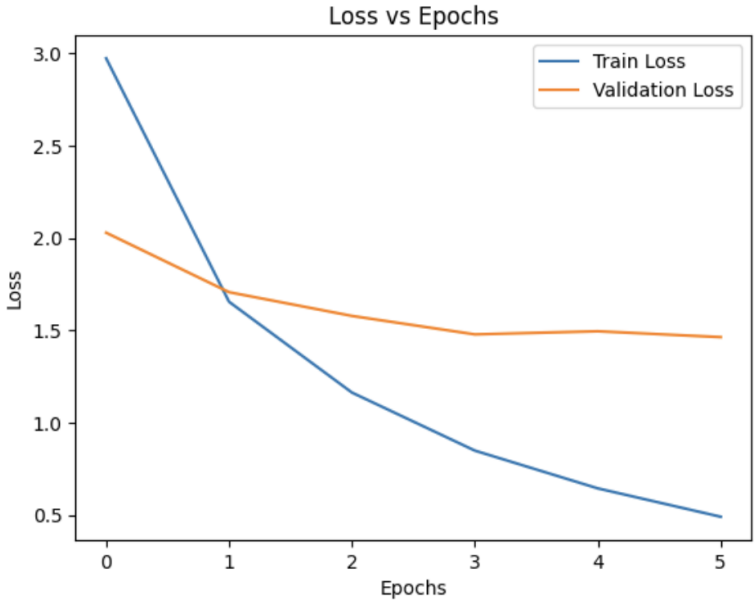

Figura 4: Evolución de la pérdida de entropía cruzada del modelo M1.

Los resultados de M1 en validación se resumen en la Tabla 2. La pérdida en el conjunto de validación se redujo de 1.74 en el modelo M0 a 1.46 en el modelo M1, lo que representa una mejora absoluta de 0.28 y relativa de aproximadamente un 16 %. Esta mejora refleja un avance en la capacidad de generalización del modelo.

Cuadro 2: Rendimiento del modelo M1 en validación

Conjunto	**Accuracy**	**Loss**
Validación	67,7 %	1,42

El modelo M1 obtiene una mejora sustancial frente a M0 gracias a la menor tasa de aprendizaje y al uso de ADAM, que permite converger más rápidamente a un mínimo de

la función de costo. La regularización L2 evita el sobre-ajuste, manteniendo la brecha entre entrenamiento y validación controlada. El early stopping detiene el entrenamiento antes de que aparezca overfitting y retiene el mejor modelo obtenido.

3.4. Modelo en PyTorch con misma configuración (M2)

Para corroborar la equivalencia de las implementaciones, se entrenó en PyTorch un modelo con la misma arquitectura e hiperparámetros de M1. La Figura 5 y la Tabla 3 muestran sus resultados.

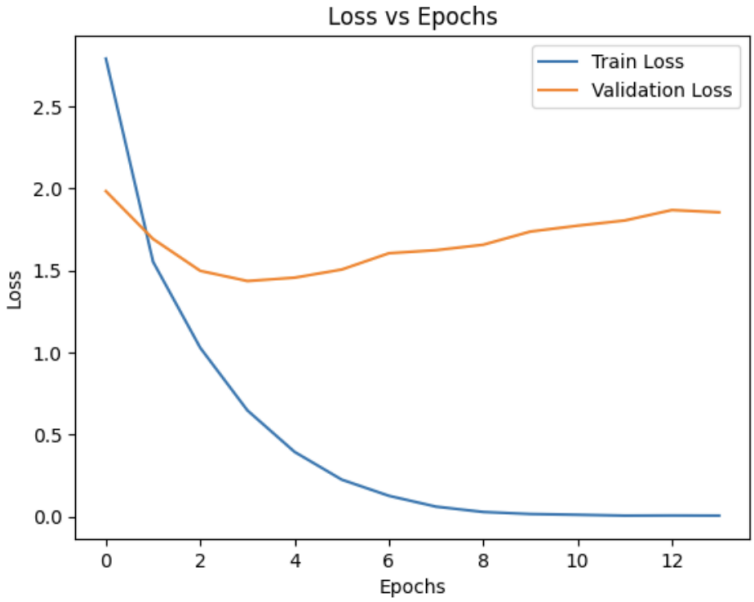

Figura 5: Evolución de la pérdida de entropía cruzada del modelo M2 (PyTorch).

Cuadro 3: Rendimiento del modelo M2 en validación

Conjunto	**Accuracy**	**Loss**
Validación	68 %	1.85

3.5. Búsqueda de arquitecturas en PyTorch (M3)

En M3 se exploraron las mismas configuraciones e hiperparámetros que en M1 mediante grid search para encontrar la mejor combinación de arquitectura. La Tabla 4 muestra el rendimiento en validación del mejor modelo encontrado.

Cuadro 4: Rendimiento del modelo M3 en validación y entrenamiento

Conjunto	**Accuracy**	**Loss**
Validación	66 %	1.4
Entrenamiento	- %	-

El mejor desempeño se obtuvo con la arquitectura $[784, 512, 256, 49]$, combinada con:

- **arquitectura**: [728,512,256,49]
- **Optimizador**: SGD
- **Dropout**: 0.5
- **Batch size**: full-batch
- **Regularización L2**: $\lambda = 0$

Esta configuración logra una *accuracy* de validación del 66 % y una pérdida de 1.4. En comparación con M1, M3 mejora su desempeño al emplear **dropout** con $p = 0{,}5$ y entrenar en *full-batch*, lo que estabiliza las actualizaciones y reduce el sobreajuste. Aunque utiliza SGD en lugar de ADAM y no aplica regularización L2, el uso de dropout y el tamaño de batch completo permiten una mejor capacidad de generalización, reflejada en una menor *validation loss* y una mayor precisión.

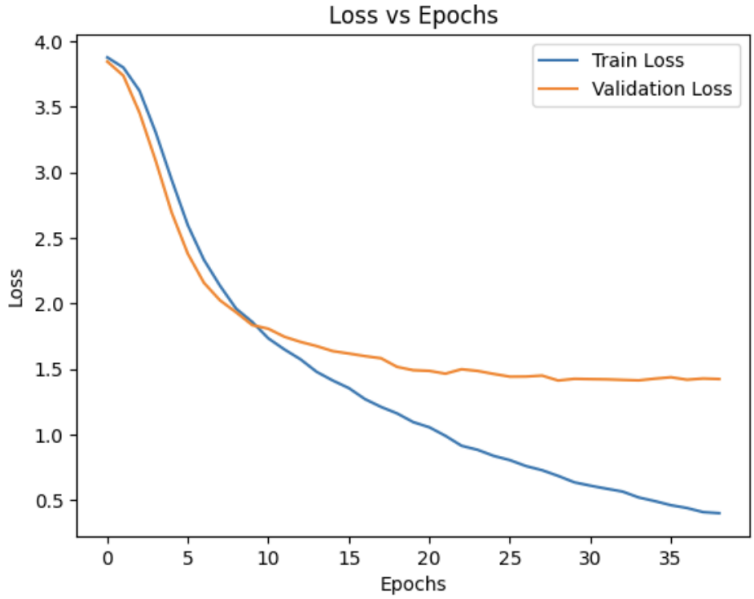

Figura 6: Evolución de la pérdida de entropía cruzada de M3 en entrenamiento y validación.

3.6. Modelo con sobre-ajuste en PyTorch (M4)

Para mostrar el efecto de sobre-ajuste, en M4 se entrenó una arquitectura de mayor capacidad sin aplicar técnicas de regularización y una mayor arquitectura, $[784, 1024, 512, 256, 49]$, con LR = 0,001 y optimizador SGD.

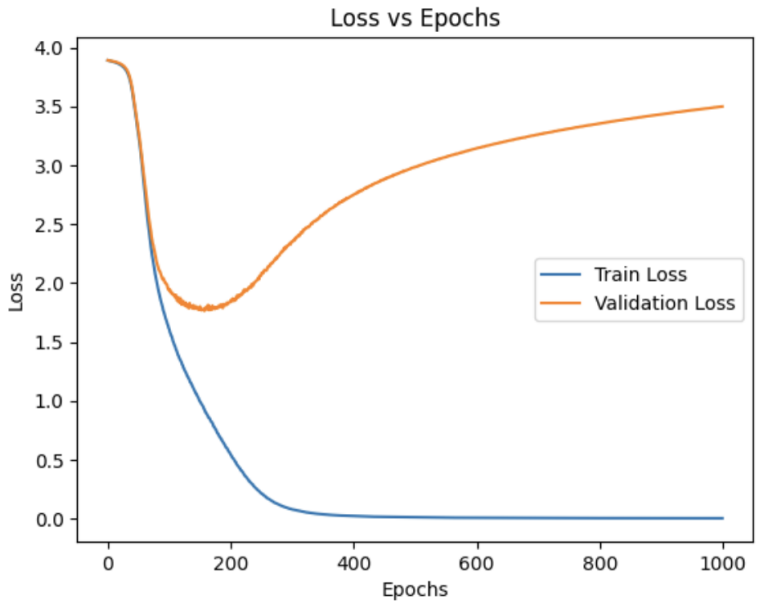

Figura 7: Evolución de la pérdida de entropía cruzada de M4 en entrenamiento y validación.

En la Figura 7 se aprecia claramente el fenómeno de sobre-ajuste. Las curvas de pérdida de entrenamiento y validación descienden en paralelo hasta aproximadamente la época 100, indicando un aprendizaje de patrones generales. A partir de ese punto, la **pérdida de entrenamiento** sigue disminuyendo casi a cero, mientras que la **pérdida de validación** comienza a aumentar, lo que evidencia que el modelo memoriza el conjunto de entrenamiento y pierde capacidad de generalización.

Cuadro 5: Rendimiento del modelo M4 en validación y entrenamiento

Conjunto	**Accuracy**	**Loss**
Validación	59.3 %	3.49
Entrenamiento	- %	-

3.7. Comparación de modelos en conjunto de prueba

La Tabla 6 muestra que el modelo M3 es el que mejor generaliza al conjunto de prueba, con un *accuracy* de 66 % y una *loss* de 1.35. Le siguen M1 (64 %, 1.45), M2 (59 %, 1.55) y M0 (58 %, 1.73), lo que evidencia la mejora progresiva al incorporar técnicas de optimización y regularización. M4, por su parte, confirma el sobre-ajuste, con *loss* muy alto en test (3.28).

Dado que M3 obtuvo las mejores métricas, este modelo fue seleccionado para el "Desafío":

Cuadro 6: Rendimiento de todos los modelos en el conjunto de prueba

Modelo	**Accuracy**	**Loss**
M0	58 %	1.73
M1	66 %	1.41
M2	67 %	1.85
M3	66 %	1.37
M4	57 %	3.28

predecir las probabilidades a posteriori de cada clase sobre el dataset `X_COMP.npy` y generar el archivo *predicciones.csv* con dichas probabilidades.